

ASSIGNMENT-8.2

NAME: SRAVANI

ROLLNO: 2303A510G7

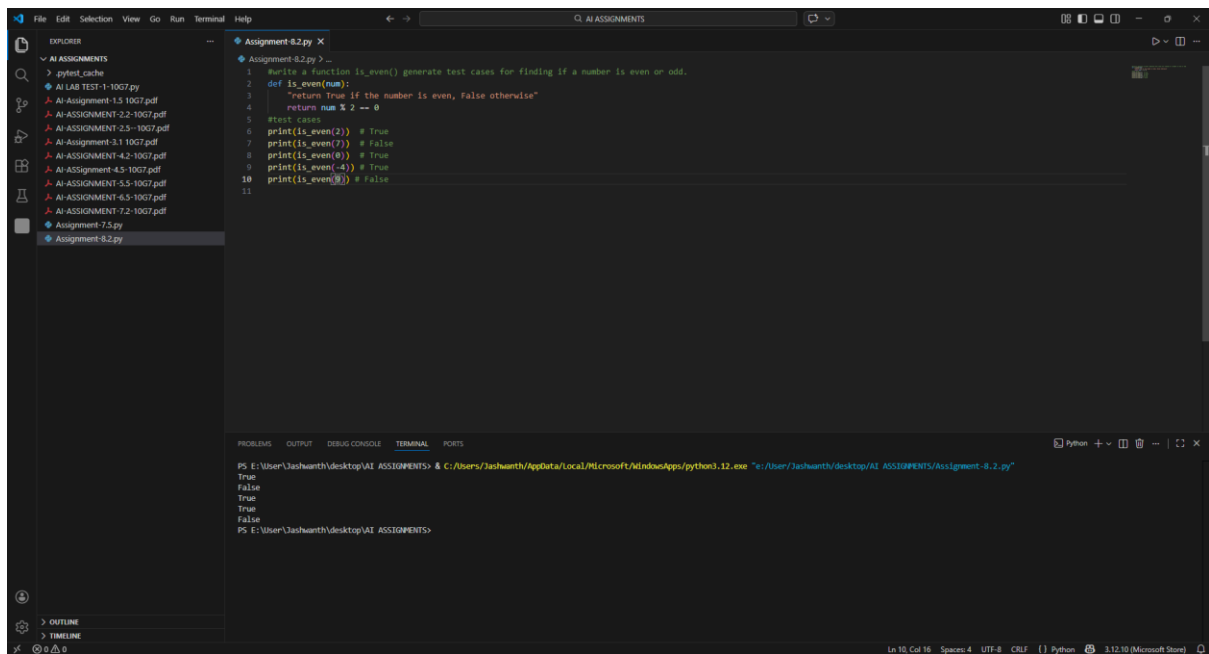
BATCH:30

TASK-1

PROMPT:

Write a function is_even() generate test cases for finding if a number is even or odd

CODE:



```
File Edit Selection View Go Run Terminal Help
AI ASSIGNMENTS
> pylest_cache
AI LAB TEST-1-10G7.py
AI-Assignment-1.5-10G7.pdf
AI-Assignment-2.2-10G7.pdf
AI-Assignment-2.5-10G7.pdf
AI-Assignment-3.1-10G7.pdf
AI-Assignment-4.2-10G7.pdf
AI-Assignment-4.5-10G7.pdf
AI-Assignment-5.5-10G7.pdf
AI-Assignment-6.5-10G7.pdf
AI-Assignment-7.2-10G7.pdf
Assignment-7.5.py
Assignment-8.2.py X

Assignment-8.2.py > ...
1 #write a function is_even() generate test cases for finding if a number is even or odd.
2 def is_even(num):
3     "return true if the number is even, false otherwise"
4     return num % 2 == 0
5 #test cases
6 print(is_even(2)) # True
7 print(is_even(7)) # False
8 print(is_even(8)) # True
9 print(is_even(-4)) # True
10 print(is_even(0)) # False
11

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - - - | : X
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\Desktop\AI ASSIGNMENTS\Assignment-8.2.py"
True
False
True
True
False
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>
```


If the function returns **True for even numbers** and **False for odd numbers**, the test passes.

3. **Exception Test Case:**

I also tested the function with invalid input, like a string.

This was to make sure the function properly raises an error when the input is not an integer.

4. **unittest Test Case:**

After that, I created a unittest class.

This helped me test multiple even and odd numbers in a structured and organized way.

5. **pytest Test Case:**

I then used pytest, where I wrote simple assert statements in separate test functions to check different inputs.

6. **Doctest:**

Finally, I added example test cases inside the function's docstring.

Using doctest, these examples were automatically executed to verify that the function works correctly.

Types of test cases:

1.Assert test case- An assert test case uses the assert keyword to automatically verify expected output.

2.normal test case- A normal test case checks output using print() and requires manual verification.

3. Exception Test Case: An exception test case checks whether a function correctly raises an error for invalid input.

4. unit test Test Case: A unittest test case uses Python's unittest module to test functions inside a class structure.

5. pytest Test Case: A pytest test case uses the pytest framework and relies on simple assert statements.

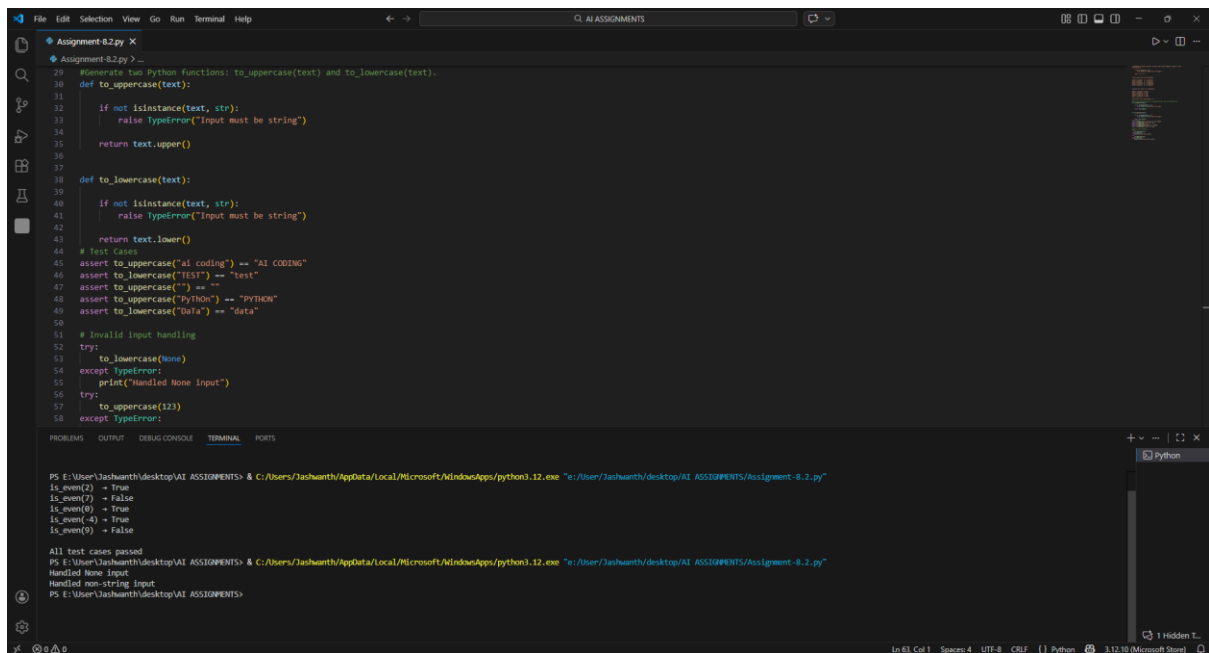
6. Doctest: A doctest is written inside a function's docstring and looks like an interactive Python session.

TASK-2

PROMPT:

Generate two Python functions: to_uppercase(text) and to_lowercase(text).

CODE:



```
File Edit Selection View Go Run Terminal Help
Assignment-8.2.py X
#Generate two python functions: to_uppercase(text) and to_lowercase(text).
def to_uppercase(text):
    if not isinstance(text, str):
        raise TypeError("input must be string")
    return text.upper()

def to_lowercase(text):
    if not isinstance(text, str):
        raise TypeError("input must be string")
    return text.lower()

# test cases
assert to_uppercase("ai coding") == "AI CODING"
assert to_lowercase("TEST") == "test"
assert to_uppercase("") == ""
assert to_uppercase("python") == "PYTHON"
assert to_lowercase("Data") == "data"

# Invalid input handling
try:
    to_lowercase(None)
except TypeError:
    print("Handled None input")
try:
    to_uppercase(123)
except TypeError:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/user/jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.2.py"
is_even(2) -> True
is_even(7) -> False
is_even(0) -> True
is_even(-4) -> True
is_even(9) -> False

All test cases passed
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/user/jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.2.py"
Handled None input
Handled non-string input
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

Both `to_uppercase()` and `to_lowercase()` functions worked correctly for valid string inputs by converting text into upper and lower case.

Assert test cases verified the outputs automatically.

When invalid inputs like `None` and numbers were given, the functions raised `TypeError`. Exception handling successfully caught these errors.

Overall, all test cases passed and the functions behaved as expected.

TASK-3

PROMPT:

Generate test cases a Python function `sum_list(numbers)` that calculates the sum of elements in a list.

CODE:

```
63 # Generate test cases a Python function sum_list(numbers) that calculates the sum of elements in a list.
64 def sum_list(numbers):
65
66     # validate input
67     if not isinstance(numbers, list):
68         raise TypeError("Input must be a list")
69
70     total = 0
71
72     for n in numbers:
73
74         # Add only numeric values
75         if isinstance(n, (int, float)):
76             total += n
77
78     return total
79
80 print("sum_list([1, 2, 3]) -> ", sum_list([1, 2, 3]))
81 print("sum_list([]) -> ", sum_list([]))
82 print("sum_list([-1, 5, -4]) -> ", sum_list([-1, 5, -4]))
83 print("sum_list([2, 'a', 3]) -> ", sum_list([2, 'a', 3]))
84 assert sum_list([1, 2, 3]) == 6
85 assert sum_list([]) == 0
86 assert sum_list([-1, 5, -4]) == 0
87 assert sum_list([2, 'a', 3]) == 5
88
89 print("All test cases passed")
90
91
92
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
is_even(0) -> True
is_even(4) -> True
is_even(9) -> False

All test cases passed
PS E:\User\Jashwanth\Desktop\VAI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\Desktop\VAI ASSIGNMENTS\Assignment-8.2.py"
Handled non-string input
PS E:\User\Jashwanth\Desktop\VAI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\Desktop\VAI ASSIGNMENTS\Assignment-8.2.py"
sum_list([1, 2, 3]) -> 6
sum_list([]) -> 0
sum_list([-1, 5, -4]) -> 0
sum_list([2, 'a', 3]) -> 5

All test cases passed
PS E:\User\Jashwanth\Desktop\VAI ASSIGNMENTS>
```

OBSERVATION:

The `sum_list(numbers)` function correctly calculated the sum of elements in a list.

It added only numeric values and ignored non-numeric data.

Test cases and assert statements verified the outputs.

Invalid input (non-list) raised a `TypeError`.

All test cases passed successfully, showing the function works as expected.

TASK-4

PROMPT:

Generate a Python class `StudentResult` with the following methods:

`add_marks(mark)`

`calculate_average()`

`get_result()`

CODE:

```
91 #Generate a Python class StudentResult with the following methods:
92
93 #add_marks(mark)
94 #calculate_average()
95 #get_result()
96
97 class StudentResult:
98
99     def __init__(self):
100         self.marks = []
101
102     # Add marks
103     def add_marks(self, mark):
104
105         # Validation: marks must be 0-100
106         if not isinstance(mark, (int, float)):
107             raise TypeError("Marks must be numeric")
108
109         if mark < 0 or mark > 100:
110             raise ValueError("Marks must be between 0 and 100")
111
112         self.marks.append(mark)
113
114     # Calculate average
115     def calculate_average(self):
116
117         if len(self.marks) == 0:
118             return 0
119
120         return sum(self.marks) / len(self.marks)
```

sum_list([2,"a",3])= 5

All test cases passed
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/user/jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.3.py"

Marks: [80, 70, 80]
Average: 76.0
Result: Pass

Marks: [30, 35, 40]
Average: 35.0
Result: Fail

Error: Marks must be between 0 and 100

All test cases passed
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>

OBSERVATION:

It contains a class named StudentResult with methods to add marks, calculate average, and get result. The code includes validation to ensure marks are numeric and between 0 and 100.

It shows marks being added, average calculated, and results displayed as Pass or Fail. An error message is also shown when invalid marks (outside 0–100) are entered. Overall, the tests appear to pass successfully.

TASK-5

PROMPT:

Generate a Python function is_valid_username(username).

CODE:

```
170 # Create a Python function is_valid_username(username).
171 def is_valid_username(username):
172
173     # Must be string
174     if not isinstance(username, str):
175         return False
176
177     # Minimum length check
178     if len(username) < 5:
179         return False
180
181     # No spaces allowed
182     if " " in username:
183         return False
184
185     # Only alphanumeric characters
186     if not username.isalnum():
187         return False
188
189     return True
190
191 print(is_valid_username("user01") == True, is_valid_username("user01"))
192 print(is_valid_username("a1") == True, is_valid_username("a1"))
193 print(is_valid_username("user name") == False, is_valid_username("user name"))
194 print(is_valid_username("user@123") == False, is_valid_username("user@123"))
195 assert is_valid_username("user01") == True
196 assert is_valid_username("a1") == True
197 assert is_valid_username("user name") == False
198 assert is_valid_username("user@123") == False
199 print("All test cases passed")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Marks: [30, 35, 40]
Average: 35.0
Result: fail

Error: Marks must be between 0 and 100

All test cases passed
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\user\jashwanth\desktop\ai ASSIGNMENTS\Assignment-8.2.py"

is_valid_username("user01") == True
is_valid_username("a1") == True
is_valid_username("user name") == False
is_valid_username("user@123") == False

All test cases passed
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>

OBSERVATION:

is_valid_username(username).

The function checks if:

- The input is a string
- The length is at least 5 characters
- There are no spaces
- It contains only alphanumeric characters

The program defines a function to check whether a username is valid. It verifies string type, minimum length, no spaces, and only letters/numbers. Test cases are executed and all outputs are correct, showing the function works properly.